

Deployment of Ubuntu Server and Services

This project describes designing and deploying a Linux-based home lab server that provides self-hosted cloud storage, secure remote access, media streaming, and network-wide ad blocking with unified internal and external access to management interfaces. The environment runs on Ubuntu Server and combines Netxccloud, Coclpit, Jellyfin, Pi-Hole, and Caddy behind a DuckDNS domain to create a cohesive, security-focused platform that is fully manageable both inside and outside the local network.

1 – Ubuntu Server on bare metal

The first step was deploying Ubuntu Server on a bare-metal, headless machine dedicated to running as a home lab server. Ubuntu Server was selected for its long-lasting support, stability, extensive package ecosystem, and strong community documentation. The server was configured for headless operation using SSH for administration, which reduce resource overhead and attack surface compared to a GUI while still allowing full remote management.

Outcome:

- Established a reliable, always-on base system to host multiple services under one platform;
- Enabled secure remote administration via SSH, preparing the foundation for service isolation, firewall configuration, and resource monitoring.

2 – LAMP stack and Nextcloud deployment

The next step was installing a LAMP stack (Linux, Apache, MariaDB/MySQL, PHP) and deploying Nextcloud as a self-hosted cloud storage and collaboration server. Apache and PHP modules were configured to meet Nextcloud's requirements, and a database was created and secured with a dedicated Nextcloud user Admin for proper separation of duties and principle of least privilege, improving security. Nextcloud was then installed and configured to handle file syncing, web-based access, and application extension for calendars, contacts, and more, completely replacing reliance on third-party cloud services.

Outcome:

- Delivered a private cloud solution with browser, desktop and mobile access for file synchronization and sharing over HTTPS;
- Centralized personal data under user control, with database-backed storage and application-level permissions.

3 – Cockpit for web-based server management

Cockpit was installed to provide a web-based interface for monitoring and managing the server's key resources, including CPU, memory, disk usage, services, and networking. Cockpit exposes a modern web UI that connects to systemd and standard Linux tooling, offering dashboards, logs, and terminal access directly in the browser. This tool was chosen to maintain full visual control over the server without needing a local display, which complements the headless Ubuntu configuration.

Outcome:

- Enabled intuitive, browser-based visibility into system health, storage utilization, network interfaces, and running services;
- Simplified routine administration tasks such as server restarts, log inspection, and resource monitoring, improving maintainability and response time to issues.

4 – Jellyfin media server deployment

After the cloud and management stack, Jellyfin was deployed on the server to provide a self-hosted media streaming platform for movies, TV shows, and other media. Jellyfin was configured to scan local, external and cloud media directories with appropriate read permissions, using the browser-based dashboard to create libraries, define content types, and tune metadata preferences. A dedicated Jellyfin user/library configuration was created to keep media paths organized and ensure the service can run continuously without interfering with other workloads.

Outcome:

- Privaded a “Netflix”/Plex-style media streaming experience that is fully open source and under local control, accessible via web and client apps on local network;

- Centralized media playback on the same physical server as the cloud and management stack, leveraging existing storage while maintaining logical separation between services.

5 – Pi-Hole for DNS-level ad-blocking and internal DNS routing

Pi-Hole was deployed on the server to act as a DNS sinkhole for the local network, blocking ad and tracker domains at the DNS level and improving privacy and bandwidth usage across all connected devices. The local router and clients were configured to use Pi-Hole as their primary DNS server, ensuring that DNS requests are filtered through Pi-Hole's blocklists and local DNS overrides. In addition to add blocking, custom DNS records were created within Pi-Hole to route “server control” hostnames (for example, internal management URLs for services for Cockpit or Nextcloud) to the appropriate internal IPs, forming the basis for clean, human- readable access patterns.

Outcome:

- Implemented network-wide ad and tracker blocking, reducing unnecessary traffic and improving user privacy across the home network;
- Established a flexible DNS layer that can map friendly hostnames to internal services.

6 – Caddy reverse proxy with DuckDNS for HTTPS inside and outside the network

A DuckDNS domain was registered and configured to point to the home network's public IP, enabling dynamic DNS to the server despite a residential connection. Caddy was then deployed as a reverser proxy, using DuckDNS domain to terminate HTTPS, automatically manage TLS certificates, and proxy requests to the internal services running on the Ubuntu Server. Caddy's internal services (such as the ones mentioned above), aligning with the DNS records created in Pi-Hole so that services are reachable via the same domain, but different sub-domains, and paths both on the LAN and from the Internet.

Outcome:

- Enabled secure HTTPS access to management and application interfaces from outside the local network, with automatic certificate provisioning and renewal for the DuckDNS domain;
- Created a coherent access model where internal devices resolve service hostnames via Pi-Hole and external clients use the DuckDNS domain through Caddy, allowing the home lab to be access and managed consistently on the local network and remotely.